

TLS

Traffic Light Simulation

Comprehensive Documentation & Performance Analysis

Version 1.0.0

Engine: SUMO 1.27 + TraCI

GUI: PyQt6 · Charts: pyqtgraph

Platform: Linux / Windows / macOS

Python 3.10 – 3.13

Table of Contents

- 1. Project Overview
- 2. Tech Stack
- 3. Project Structure
- 4. Quick Start
- 5. Git Workflow & Contribution
- 6. Code Architecture (MVC)
- 7. Performance Analysis
- 8. Bottleneck Deep Dive
- 9. Fix Priority & Responsibility
- 10. Build & Deploy
- 11. Test Strategy
- 12. Appendix: Key File Reference

1. Project Overview

TLS (Traffic Light Simulation) is a desktop application for city-scale traffic simulation using the **SUMO** (Simulation of Urban MObility) engine. It provides a real-time interactive GUI with multiple traffic light algorithms, live metrics dashboard, and OSM map import support.

Key Features

- 3 built-in map presets: Pamulang (Indonesia), Silicon Valley (USA), Tokyo (Japan)
- 4 traffic light algorithms: Fixed-Time, Actuated, Green Wave, Max-Pressure
- Real-time dashboard: avg speed, wait time, throughput, queue length, fuel, CO₂
- OSM import: download any city from OpenStreetMap and simulate instantly
- CSV / JSON export for post-analysis
- Multi-platform: Linux, Windows, macOS (via Python + PyQt6)

Problem Statement

Traditional traffic light timing is static and inefficient, causing congestion, fuel waste, and pollution. TLS enables researchers and city planners to test adaptive TL algorithms (Actuated, Max-Pressure, Green Wave) against real-world or synthetic traffic scenarios before deployment.

Target Audience

- Traffic engineers evaluating adaptive signal control
- Researchers studying urban mobility
- Students learning SUMO/TraCI simulation
- City planners comparing scenarios

2. Tech Stack

TLS is built on a Python foundation with a real-time coupling to the SUMO traffic simulator via the TraCI protocol. The GUI layer uses PyQt6 with pyqtgraph for live charting.

Component	Technology	Version
Simulation Engine	SUMO (via TraCI)	1.20+ (1.27 tested)
GUI Framework	PyQt6	>= 6.5
Real-time Charts	pyqtgraph	>= 0.13
PDF Export (optional)	weasyprint	>= 60
Language	CPython	3.10 - 3.13
Database	SQLite (stdlib)	built-in
OS (Dev)	Ubuntu 22.04 / Windows 11	—

Why SUMO?

SUMO is the de-facto open-source traffic simulation engine (DLR, since 2001). It supports large-scale networks (entire cities), multi-modal transport, emission modeling, and TraCI — a TCP-based protocol for real-time control. TLS uses TraCI to step the simulation, query vehicle positions, and set traffic light phases on every step.

Why PyQt6?

PyQt6 provides native desktop performance with hardware-accelerated rendering (QGraphicsView for the map, QTimer-based animation loop). pyqtgraph leverages PyQt6's OpenGL support for smooth real-time chart updates at 30+ FPS.

Python 3.14 Warning

Python 3.14 **does not have wheels** for PyQt6 or pyqtgraph yet. The setup scripts (setup.bat, setup.ps1, setup.sh) will warn if Python 3.14 is detected. Use Python 3.10–3.13.

3. Project Structure

The project follows a clean MVC-like layout within `app/`, with simulation data under `sim/` and utilities under `scripts/`:

```
traffic-light-sim/
+-- app/
|   +-- main.py                Entry point
|   +-- engine/
|   |   +-- traci_client.py    TraCI wrapper (Model)
|   |   +-- sim_controller.py  Simulation loop (Controller)
|   |   +-- tl_algorithms.py   TL algorithm functions
|   |   +-- osm_importer.py    OSM -> SUMO netimport
|   +-- gui/
|   |   +-- map_viewer.py      Map + vehicles + TL (QGraphicsView)
|   |   +-- main_window.py     Window layout, menus, signals
|   |   +-- dashboard.py       Real-time charts (pyqtgraph)
|   |   +-- controls.py        Play/Pause/Speed toolbar
|   |   +-- config_panel.py    Algorithm config panel
|   |   +-- settings_dialog.py  App settings
|   |   +-- scenario_dialog.py  Save/load scenarios
|   |   +-- tile_provider.py    OSM tile background
|   +-- models/
|   |   +-- traffic_light.py    TLPhase, TrafficLight dataclasses
|   |   +-- vehicle.py         Vehicle dataclass
|   +-- metrics/
|   |   +-- collector.py        MetricsCollector (ring buffer)
|   |   +-- storage.py          SQLite/JSON persistence
|   +-- utils/
|   |   +-- config.py           Config manager (JSON file)
|   |   +-- localization.py     i18n (id_ID, en_US)
|   |   +-- logger.py           Loguru wrapper
+-- sim/
|   +-- pamulang/              Pamulang map data
|   +-- silicon_valley/        Silicon Valley map data
|   +-- tokyo/                 Tokyo map data
+-- scripts/
|   +-- setup_maps.py          Initialize map folders
|   +-- generate_logo.py       Generate icon.png + icon.ico
|   +-- generate_docs_pdf.py   This PDF generator
|   +-- add_tls_to_network.py  Utility for TL placement
+-- tests/
+-- .github/workflows/build.yml CI/CD pipeline
+-- setup.bat                 Windows (CMD) setup
+-- setup.ps1                 Windows (PowerShell) setup
+-- setup.sh                  Linux/macOS setup
+-- tls.spec                  PyInstaller spec
+-- requirements.txt           Python dependencies
+-- README.md                 Project readme
```

4. Quick Start

Prerequisites

- Python 3.10 – 3.13 (**3.14 NOT supported** — no PyQt6 wheels)
- SUMO 1.20+ installed and in PATH (or set SUMO_HOME env var)

Clone & Setup

```
# Linux / macOS
git clone https://github.com/Cefneal/traffic-light-sim.git
cd traffic-light-sim
bash setup.sh
source venv/bin/activate && python -m app.main

# Windows (PowerShell - recommended)
git clone https://github.com/Cefneal/traffic-light-sim.git
cd traffic-light-sim
.\setup.ps1
.\venv\Scripts\Activate.ps1 ; python -m app.main

# Windows (CMD)
setup.bat
venv\Scripts\activate & python -m app.main
```

Pre-built Executable

Download from [GitHub Releases](#) (auto-built via GitHub Actions on every tag push):

```
# Linux
wget https://github.com/Cefneal/traffic-light-sim/releases/latest/download/TLS-Linux-x64.tar.gz
tar xzf TLS-Linux-x64.tar.gz
cd TLS-Linux-x64 && ./tls

# Windows - download TLS-Windows-x64.zip from Releases, extract and run tls.exe
```

NOTE: SUMO must still be installed separately — it is too large to bundle (~200 MB).

5. Git Workflow & Contribution

Branching Strategy

The project follows a simplified trunk-based development on main with feature branches:

- **main** — Stable, always deployable. Protected branch.
- **feature/*** — For new features and fixes. Merge via PR.
- **fix/*** — For urgent hotfixes (P1 items).

Daily Workflow

```
# 1. Pull latest
git checkout main
git pull origin main

# 2. Create feature branch
git checkout -b feature/my-fix

# 3. Work, commit frequently
git add app/engine/sim_controller.py
git commit -m "fix: reduce TraCI calls in simulation loop"

# 4. Push branch
git push -u origin feature/my-fix

# 5. Create PR on GitHub, get reviewed
# 6. Merge to main after approval
```

Commit Message Convention

Prefix	Meaning	Example
fix:	Bug fix	fix: race condition in TraCI thread access
feat:	New feature	feat: add OSM import dialog
perf:	Performance	perf: cache vehicle subscription results
docs:	Documentation	docs: add Windows setup instructions
refactor:	Code change	refactor: extract TL building logic
chore:	Maintenance	chore: update requirements.txt pins

Tagging & Release

Tags trigger automated builds via GitHub Actions:

```
# List tags
git tag -l

# Create and push a new release tag
git tag -a v1.0.0 -m "Release v1.0.0"
git push origin v1.0.0

# This triggers .github/workflows/build.yml:
#   - Test: pytest on linux-latest
#   - Build Linux: PyInstaller + bundle SUMO binaries
#   - Build Windows: PyInstaller + download SUMO portable
#   - Release: Upload all artifacts to GitHub Releases
```

GitHub Actions Pipeline

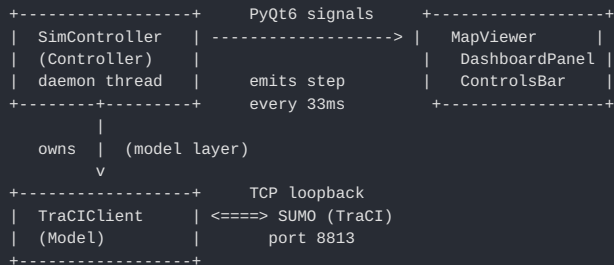
The CI/CD pipeline (`.github/workflows/build.yml`) has 4 jobs:

- **test** — Runs pytest on ubuntu-latest with SUMO installed via apt
- **build-linux** — PyInstaller build, bundles SUMO binaries, uploads artifact
- **build-windows** — Uses windows-latest runner, downloads SUMO portable zip, bundles
- **release** — On tag push, publishes all artifacts to GitHub Releases

6. Code Architecture (MVC)

TLS follows a classic Model-View-Controller pattern:

- **Model** — TraCIClient wraps all TraCI calls, manages subscriptions, caches data
- **Controller** — SimController runs the simulation loop in a daemon thread
- **View** — PyQt6 widgets (MapView, DashboardPanel, ControlsToolbar)



6.1 TraCIClient (Model)

Located in `app/engine/traci_client.py` (499 lines). Wraps all TraCI calls and provides subscription-based cached access.

Key design decisions:

- **Subscriptions:** Edges, vehicles, and traffic lights are subscribed once (or incrementally) so that after each `simulationStep()`, cached data can be read with zero additional TraCI socket calls.
- **Caches:** `_cached_tl_ids` stores TL IDs to avoid repeated `getIDList()` calls.
- **Thread-unsafe (ISSUE):** All methods call `traci.*` directly without locks. This is the #1 cause of `setPhase` failed errors when the GUI thread reads while the sim thread writes.

```
# CORRECT usage (sim thread only):
tc.subscribe_edges(all_edges)      # one-time
tc.simulation_step()               # advances SUMO
data = tc.get_edge_data_cached(eid) # zero socket calls

# WRONG (causes 14,000+ calls/step):
tc.get_total_fuel_consumption()     # loops all vehicles!
```

6.2 SimController (Controller)

Located in `app/engine/sim_controller.py` (298 lines). Runs the simulation loop in a daemon thread.

Responsibilities:

- Start/stop/pause/resume simulation (SUMO process lifecycle)
- Step timing: calls `simulation_step()`, dispatches TL algorithms, collects metrics
- Event system: emits `step`, `start`, `stop` events to GUI listeners via PyQt6 signals

```
# Simulation loop (sim_controller.py:179-248)
while self._running:
    if not self._paused:
        veh_ids = self.traci.get_vehicle_ids()
        self.traci.subscribe_vehicles(veh_ids) # BUG: leaks subscriptions
        self.traci.simulation_step()
        self._algorithm_fn(tl, traci, sim_time) # dispatches TL algo
        self.collector.record(...)
        self._emit('step', data)
        time.sleep(1.0 / (30 * speed))         # BUG: no drift correction
```

6.3 GUI (View)

Three main widgets update independently:

- **MapView** (674 lines): QGraphicsView with QTimer-based animation loop at ~30 FPS. Renders vehicles, roads, junctions, TL lights, trails, heatmap, OSM tiles.
- **DashboardPanel** (181 lines): pyqtgraph PlotWidget with rolling time-series charts (speed, wait time, throughput, queue, fuel, CO₂).

- **ControlsToolBar** (~150 lines): Play, Pause, Stop, Speed slider, scenario dropdown.

6.4 Data Flow Per Step

Step	Thread	Action	TraCI Calls
1	Sim	get_vehicle_ids() + subscribe_vehicles()	2+
2	Sim	simulationStep() — advances SUMO 1 step	1
3	Sim	get_all_vehicles_cached() — reads subscription	0
4	Sim	get_total_fuel_consumption() — loops ALL vehicles	N*2
5	Sim	TL algorithms — edge calls, setPhase	10-1500
6	Sim	Emit 'step' signal to GUI	0
7	GUI	get_all_vehicles_cached() — reads subscription	0+
8	GUI	get_cached_tl_state() per TL — subscription	0

7. Performance Analysis

This section identifies all performance bottlenecks categorized by severity. The TraCI protocol communicates over TCP loopback — each call has ~100µs latency. With **14,000+ calls per step** and a 30 FPS target, the simulation spends most of its time waiting for TraCI responses rather than computing.

P1 - Critical (must fix immediately)

These issues cause significant lag, crashes, or incorrect behavior. They block basic usability.

#	Issue	Location	Impact	Calls/Step
1.1	Fuel/CO ₂ loop per vehicle	traci_client.py:401-423	N vehicles x 2 calls/step	~10,000
1.2	MaxPressure edge query loop	tl_algorithms.py:84-116	N edges x N phases x N TLs	~1,500
1.3	Thread race condition (GUI+SIM)	sim_controller + map_viewer	Crash: "setPhase failed / Connection closed"	N/A

P2 - High (should fix soon)

#	Issue	Location	Impact	Calls/Step
2.1	Vehicle subscription leak	sim_controller.py:189-192	SUMO slows progressively; memory grows	N/A
2.2	Double getIDList() call	sim_controller + traci_client	Redundant ID fetch per step	~2
2.3	Sleep-based timing drift	sim_controller.py:245-246	Real-time sync degrades over time	0
2.4	step_single missing step_length arg	sim_controller.py:259	actuated_controller gap_timer broken	0

P3 - Medium (nice to have)

#	Issue	Location	Impact	Calls/Step
3.1	TL get_tl_ids called twice per frame	map_viewer.py:526,543	Double ID fetch per GUI update	~2
3.2	Dashboard chart re-render 30 Hz	main_window.py:114-128	CPU waste on chart updates	0
3.3	Vehicle cleanup full dict scan O(n)	map_viewer.py:508-519	Loop over all items every frame	0
3.4	p.index vs p.next bug in get_tl_program	traci_client.py:301	Wrong phase index stored	0

Total estimated impact: ~14,000+ TraCI calls/step before fixes → ~50-200 after fixes. FPS should stabilize at 30 from 8-12.

8. Bottleneck Deep Dive

8.1 Fuel/CO₂ Loop (P1 #1.1)

The `get_total_fuel_consumption()` and `get_total_co2_emission()` methods each iterate over all vehicle IDs and call `getFuelConsumption(vid)` / `getCO2Emission(vid)` individually. With 5,000 vehicles, this is 10,000 TraCI calls per step. At 30 FPS, this is 300,000 calls/second.

```
# traci_client.py:401-423
def get_total_fuel_consumption(self) -> float:
    total = 0.0
    for vid in traci.vehicle.getIDList():          # 5000 ids
        total += traci.vehicle.getFuelConsumption(vid) # 5000 calls
    return total

# Called every step in sim_controller.py:208-209
total_fuel = self.traci.get_total_fuel_consumption() # 5000 calls
total_co2 = self.traci.get_total_co2_emission()      # 5000 calls
```

FIX: Add `VAR_FUELCONSUMPTION` and `VAR_CO2EMISSION` to the vehicle subscription variables list. Then read from subscription results instead. Alternatively, compute fuel/CO₂ only every 10th step since they change slowly.

8.2 Max-Pressure Edge Query (P1 #1.2)

The `max_pressure_controller` queries edge data for every TL phase individually instead of using the already-subscribed edge data:

```
# tl_algorithms.py:96-108
edge_ids = traci.module.edge.getIDList()[:50] # 1 call
for i, phase in enumerate(tl.phases):          # ~3 phases
    for eid in edge_ids:                       # ~50 edges
        count = traci.module.edge.getLastStepVehicleNumber(eid) # 1 call
        speed = traci.module.edge.getLastStepMeanSpeed(eid)    # 1 call
    # Total per TL: 1 + 3 * 50 * 2 = 301 calls per TL
```

With ~10 TLs, `max_pressure` alone accounts for ~3,000 calls/step.

FIX: Use `get_edge_data_cached(eid)` which reads from the subscription results instead of making new TraCI calls. Edge data is already subscribed in `sim_controller.py`:164-165.

8.3 Thread Race Condition (P1 #1.3)

This is the most critical — and the hardest to reproduce — bug. Two threads access the same TraCI connection simultaneously:

```
# THREAD 1: SimController._run_loop (sim thread)
self.traci.simulation_step()          # TraCI: SimulationStep command
# during this call, the socket is in use

# THREAD 2: MapViewer._update_view (GUI thread, ~30 Hz)
vehicles = tc.get_all_vehicles_cached() # TraCI: getIDList + subscription results
# This fires while Thread 1 has the socket locked!

# Result:
# - "setPhase failed: Connection already closed"
# - Random crashes
# - GUI freezes waiting for TraCI lock
```

FIX: Implement a thread-safe data exchange pattern. The sim thread writes vehicle/TL data to a shared buffer protected by `threading.Lock`. The GUI thread reads from this buffer. **Never touch traci.* from the GUI thread.**

8.4 Vehicle Subscription Leak (P2 #2.1)

Every step, the simulation loop subscribes to all current vehicle IDs. Vehicles that leave the network are never unsubscribed:

```
# sim_controller.py:189-192
veh_ids = self.traci.get_vehicle_ids()
self.traci.subscribe_vehicles(veh_ids)

# traci_client.py:158-167
new_vids = [vid for vid in vehicle_ids if vid not in self._subscribed_vehicles]
```

```
for vid in new_vids:
    traci.vehicle.subscribe(vid, ...) # ADD but never REMOVE
    self._subscribed_vehicles.add(vid) # grows unbounded!
```

Over a 1-hour simulation with 10,000 unique vehicles passing through, SUMO accumulates 10,000 active subscriptions — each processed on every `simulationStep()`, causing progressive slowdown.

FIX: After each step, compute set difference: `gone = subscribed - current_ids`, then unsubscribe gone vehicles via `traci.vehicle.unsubscribe(vid)`.

9. Fix Priority & Responsibility

Fixes should be implemented in priority order. Each P1 fix is independent and can be done in parallel.

Sprint 1: P1 - Critical (estimated 2-3 days)

#	Task	Effort	Files	Owner
1.1	Remove Fuel/CO ₂ loop; read from subscription or compute every 10 steps	4h	traci_client.py, sim_controller.py	Backend
1.2	Rewrite max_pressure to use get_edge_data_cached()	3h	tl_algorithms.py	Algorithm
1.3	Add threading.Lock around TraCI; buffer vehicle/TL data for GUI	8h	traci_client.py, map_viewer.py	Backend+GUI

Sprint 2: P2 - High (estimated 2 days)

#	Task	Effort	Files	Owner
2.1	Unsubscribe vehicles that have left the network	3h	traci_client.py, sim_controller.py	Backend
2.2	Cache vehicle IDs to avoid double getIDList()	1h	traci_client.py	Backend
2.3	Fix sleep timing: actual_sleep = target - elapsed_body_time	2h	sim_controller.py	Backend
2.4	Add missing step_length arg to step_single()	1h	sim_controller.py	Backend

Sprint 3: P3 - Medium (estimated 1 day)

#	Task	Effort	Files	Owner
3.1	Cache tl_ids after lazy init in map_viewer	0.5h	map_viewer.py	GUI
3.2	Throttle dashboard updates to 4-5 Hz (use QTimer or counter)	1h	main_window.py	GUI
3.3	Use set difference for vehicle cleanup (gone = old - new)	1h	map_viewer.py	GUI
3.4	Fix p.next -> p.index in get_tl_program()	0.5h	traci_client.py	Backend

Expected Impact After Fixes

Metric	Before	After (estimated)
TraCI calls per step	~14,000+	~50-200
GUI FPS (1000 vehicles)	8-12 (adaptive)	30 (stable)
Thread safety	Crashes	Stable with shared buffer
Memory (1h simulation)	Growing (sub leak)	Stable
Sim timing drift	Significant	< 1% error

10. Build & Deploy

PyInstaller Executable

TLS uses PyInstaller to create a standalone executable. The spec file (`tls.spec`) bundles the app, simulation data, resources, and all Python dependencies into a single binary.

```
# Install PyInstaller
pip install pyinstaller

# Build
pyinstaller tls.spec --clean -y

# Output:
#   dist/tls/tls          (Linux)
#   dist/tls/tls.exe      (Windows)
#   dist/TLS.app/         (macOS, if sys.platform == 'darwin')
```

GitHub Actions Automation

The `.github/workflows/build.yml` runs on every push to main and on tag push (`v*`):

```
# Pipeline (simplified):
jobs:
  test:      pytest on ubuntu-latest (with SUMO via apt)
  build-linux: PyInstaller + bundle SUMO binaries
  build-windows: PyInstaller + download SUMO portable zip
  release:   Upload artifacts to GitHub Releases

# To trigger a release:
git tag -a v1.0.0 -m "Release v1.0.0"
git push origin v1.0.0
```

SUMO Bundling Strategy

The executables do NOT bundle SUMO (~200 MB, system-dependent):

- **Linux:** Workflow installs sumo via apt, copies bin/sumo to the bundle
- **Windows:** Workflow downloads SUMO portable zip (official), extracts to bundle
- **macOS:** Requires brew `install sumo` (not bundled yet)

The app searches for SUMO in: `PATH`, `SUMO_HOME`, and uses `shutil.which()` for reliable detection on all platforms.

tls.spec Notes

- `console=True`: Shows a terminal window on Windows for debug output
- `BUNDLE` block is macOS-only (conditional on `sys.platform == 'darwin'`)
- `upx=True`: Compresses the executable with UPX (reduces size ~30%)
- Hidden imports include all TraCI domain modules, PyQt6 submodules, xml/json/csv

11. Test Strategy

Test Levels

Level	Scope	Tool	Frequency
Unit	Individual functions (TraCI wrapper, algorithms)	pytest	Every commit
Integration	SimController + TraCIClient (end-to-end step)	pytest + SUMO	Every PR
Performance	TraCI calls/step count, FPS benchmark	pytest-benchmark	Weekly
Regression	TL behavior, dashboard update, export	pytest (headless)	On P1 fix

Planned Test Cases

- `TestFixtureTime`: TL switches phase at correct intervals
- `TestActuated`: Detector triggers extension logic
- `TestMaxPressure`: Phase selected based on edge load
- `TestGreenWave`: Phase sync offset calculation
- `TestFuelMetrics`: Fuel/CO₂ values within expected range
- `TestThreadSafety`: No crashes after 1000 steps with concurrent GUI reads
- `TestSubLeak`: Subscribed vehicle count stabilizes after warmup

Regression Test After Each P1 Fix

```
# 1. Smoke test: start sim, run 100 steps, stop
python -c "from tests.smoke import *; test_smoke()"

# 2. Metric comparison: TraCI calls before/after
python -c "from tests.traci_count import *; compare_calls()"

# 3. Thread safety: loop with concurrent GUI reads
python -c "from tests.thread_safety import *; test_no_crash(1000)"

# 4. Full test suite
python -m pytest tests/ -v
```

12. Appendix: Key File Reference

Engine Layer

File	Key Classes/Functions	Lines
traci_client.py	TraCIClient: connect, subscribe, cached reads, TL control	499
sim_controller.py	SimController: start, stop, _run_loop, step_single	298
tl_algorithms.py	fixed/actuated/max_pressure/green_wave + _switch_to_next	173
osm_importer.py	OSMImporter: convert .osm to .net.xml via netconvert	~80

GUI Layer

File	Key Classes/Functions	Lines
main_window.py	MainWindow: layout, menus, signal bridge, sim lifecycle	233
map_viewer.py	MapView: network rendering, vehicles, TL, heatmap, tiles	674
dashboard.py	DashboardPanel: pyqtgraph charts, CSV/JSON export	181
controls.py	ControlsToolbar: play/pause/stop, speed, scenario selector	~150

Metrics & Models

File	Key Classes/Functions	Lines
collector.py	MetricsCollector: ring buffer, record, summary stats	83
storage.py	StorageManager: SQLite + JSON persistence	~130
traffic_light.py	TLPhase, TrafficLight: phase state, elapsed time	~40
vehicle.py	Vehicle: position, speed, angle, type dataclass	~30

Setup & Config

File	Purpose
setup.bat	Windows CMD setup script
setup.ps1	Windows PowerShell setup script (recommended)
setup.sh	Linux/macOS setup script
tls.spec	PyInstaller spec for building executables
requirements.txt	Python dependency list
.github/workflows/build.yml	CI/CD pipeline configuration
README.md	Project documentation (quick start, troubleshooting)